Sistemas de Automatización de Oficinas (SAO)

Unidad 5 — Estructuras de datos: Arreglos, Matrices y Cadenas

Objetivos de la guía

Esta guía te adentra en el mundo de las estructuras de datos estáticas:

- Uso de arreglos unidimensionales (vectores).
- Recorridos, búsquedas, corrimientos y eliminación de elementos en vectores.
- Uso de arreglos bidimensionales (matrices) y vectores paralelos.

PARTE A: Vectores (Arreglos Unidimensionales)

Ejercicio 1 — Carga y sumatoria de vector

> 💡 **Concepto:** Para recorrer un arreglo en Java, solemos usar un ciclo `for` desde `0` hasta `arreglo.length - 1`.

Creá un vector de números enteros de tamaño 5. Utilizando un ciclo `for`, solicitá al usuario ingresar cada número por teclado. Luego, utilizá otro ciclo para recorrer el vector y calcular la sumatoria total de sus elementos. Imprimí el resultado.

Ejercicio 2 — Todos iguales / Buscar mayor

1. Escribir un método `boolean todos_iguales(int[] vector)` que indique si todos los elementos de un vector son idénticos entre sí. *(Ej: `[6,6,6]` → *true*)*.
2. Escribir la función `int mayor(int[] arreglo, int inf, int sup)`, que reciba un arreglo y dos valores índices (`inf` y `sup`). La función debe regresar el mayor valor del arreglo **solamente** buscando dentro de ese rango `[inf, sup]`.

Ejercicio 3 — Normalizar vector

Para un vector `X` de `N` elementos numéricos reales, informar el mismo vector pero *normalizado*. Cada componente `X[I]` se debe reemplazar aplicando la siguiente fórmula matemática:

```
NuevoX[I] = (X[I] - Min) * 100 / (Max - Min)
```

(Donde `Min` es el valor mínimo de todo el vector y `Max` el máximo. Deberás buscar esos valores primero).

Ejercicio 4 — Intercambiar elementos

Escribir una función que permita intercambiar dos elementos cualesquiera dentro de un vector de N elementos, pasando como parámetros de entrada los elementos a intercambiar. (Recordá verificar primero si los elementos realmente existen dentro del vector para poder realizar el intercambio).

Ejercicio 5 — Movimiento de elementos (Rotar vector)

> 💡 **Sugerencia:** Para no perder el valor del primer o último elemento al desplazar los demás, recordá guardarlo en una variable auxiliar.

1. **Rotar a la izquierda:** Escribí una función que rote una posición a la izquierda todos los elementos del vector. El elemento que sale de la posición cero pasa a ser el último. *(Ej: `[1,2,3,4]` → `[2,3,4,1]`)*
2. **Rotar a la derecha:** Escribí otra función que mueva el último elemento a la posición cero, empujando los demás a la derecha. *(Ej: `[1,2,3,4]` → `[4,1,2,3]`)*

Ejercicio 6 — Suma de dígitos iterativa

Se leen 10 valores enteros menores que 30 por teclado (guardalos en un vector). Luego, se comienzan a leer valores enteros positivos de forma indefinida hasta que ocurra un evento: que la **suma de los dígitos** del número que acabás de leer sea igual a alguno de los 10 valores inicialmente guardados en el vector. Al ocurrir esto, informá cuál fue el número que disparó el corte.

(Ejemplo: Si el vector tiene `[1, 6, 25...]` y se ingresa el número `694`, sus dígitos suman 6+9+4=19. ¿Está el 19 en el vector? Si no está, seguís pidiendo).

Ejercicio 7 — Inserción en vector estático

> 💡 Insertar en vectores estáticos requiere "hacer espacio" primero, moviendo los elementos de la derecha hacia el final del vector (corrimiento).

Una función debe recibir un vector de números enteros e insertar un elemento *justo detrás* de cada elemento que tenga un valor múltiplo de 100. El elemento a insertar debe ser `valor + 1`. *(Asumí que el arreglo tiene un tamaño físico lo suficientemente grande para soportar las inserciones).*

Ejercicio 8 — Eliminación de repetidos

> 💡 Para eliminar en vectores estáticos sin dejar "huecos", debés sobreescribir el elemento a borrar desplazando todos los elementos de su derecha una posición hacia la izquierda.

Una función debe recibir un vector **ordenado** de números enteros positivos. La función debe modificar el vector *in situ* eliminando los valores repetidos.

Ejercicio 9 — Eliminación con condición compleja

Una función debe recibir un vector de números enteros positivos y retornar los datos actualizados, eliminando del vector **todos los valores impares que se encuentren atrapados entre dos valores pares adyacentes**. *(Ej: Si el vector tiene* `{1, 2, 3, 4, 7, ...}`*, el 3 es impar y está entre el 2 y el 4 (pares). Debe ser eliminado).*

PARTE B: Matrices

Ejercicio 10 — Matriz simple (Correos)

Una empresa posee 5 servidores y desea analizar el tráfico de correo electrónico entre ellos. Almacená los datos en una matriz `correo` de 5x5. El elemento `correo[i][j]` indicará el número de Kb de correo enviados desde el servidor `i` hacia el servidor `j`. Obtener:

1. Total de Kb enviado por cada uno de los servidores (sumatoria por filas).
2. Número del servidor que ha enviado mayor cantidad total de correos.
3. Mostrá qué servidores recibieron más de `M` Kb en total (sumatoria por columnas), donde `M` es ingresado por teclado.

Ejercicio 11 — Aerolíneas Nacionales (Vectores Paralelos y Matrices)

💡 Requiere cruzar información de una matriz bidimensional con información de un vector.

Una aerolínea posee aviones de 4 categorías (0, 1, 2, 3) y viaja a 50 localidades. Se dispone de una **matriz de pasajes vendidos**, indexada por código de categoría (filas) y código de destino (columnas). Además, existe un **vector de nombres de localidades**, ordenado de forma tal que la posición coincida con la columna de la matriz.

Se pide informar:

1. La cantidad de pasajeros por localidad.
2. La cantidad de pasajeros por categoría.
3. Las localidades a las que no se vendió ningún pasaje.
4. La localidad (su nombre) a la que viajó la menor cantidad de pasajeros.
5. La cantidad de localidades que superaron los 1000 pasajeros.

6. Indicar si existe o no alguna categoría de aviones para las que no se vendió *absolutamente ningún* pasaje en todos sus destinos.
